

Spring Framework

An introduction to the most powerful and widely-used Java backend framework — built to simplify enterprise development and accelerate your journey as a Full Stack Developer.



Simplicity

Less boilerplate code

Productivity

Faster development

Reliability

Enterprise-grade architecture

THE PROBLEM

Why Do We Need Spring?

Traditional Java (J2EE) Challenges

- Heavy, verbose configuration
- Tight coupling between components
- Difficult to test and maintain
- Slow development cycles

What Enterprise Apps Actually Need

- Scalability to handle growing demand
- Maintainability over the long term
- Clean, modular architecture

- ✓ ☒ Spring simplifies enterprise Java development — removing the pain of traditional J2EE while delivering everything modern apps require.

OVERVIEW

What is Spring Framework?

Spring is an open-source, lightweight, and modular Java framework designed to make enterprise application development faster, cleaner, and more productive.

What You Can Build

- Web applications
- Microservices
- Backend systems

Core Focus

- Reduce boilerplate code
- Improve developer productivity
- Promote clean design patterns

Key Characteristics

- Open-source & community-driven
- Lightweight footprint
- Highly modular structure

CORE PHILOSOPHY

The Core Idea

"Focus on business logic, not plumbing code"

Spring Handles the Plumbing

- Object creation and lifecycle management
- Dependency management between components
- Application configuration and wiring

You Focus on What Matters

- Building features users need
- Implementing business requirements
- Delivering value faster

Key Concepts

Three foundational principles underpin everything Spring does. Understanding these is essential to mastering the framework.



IoC — Inversion of Control

The framework takes control of the object lifecycle. Instead of your code creating objects, Spring manages their creation, configuration, and destruction automatically.



DI — Dependency Injection

Dependencies between components are injected automatically by the Spring container. No manual wiring required — just declare what you need and Spring provides it.



AOP — Aspect-Oriented Programming

Handles cross-cutting concerns such as logging, security, and transaction management — keeping them separate from your core business logic for cleaner code.

Benefits of Spring

Spring delivers a compelling set of advantages that make it the go-to choice for Java backend development across the industry.



Loose Coupling

Components are independent of each other, making the system flexible and easy to change without cascading side effects.



Better Testability

Dependency injection makes it straightforward to swap real implementations with mocks, enabling thorough unit and integration testing.



Modular Architecture

Use only the modules you need. Spring's modular design means you're never forced to include unnecessary dependencies.



Easier Maintenance

Clean separation of concerns and loose coupling make long-term maintenance significantly less painful and costly.

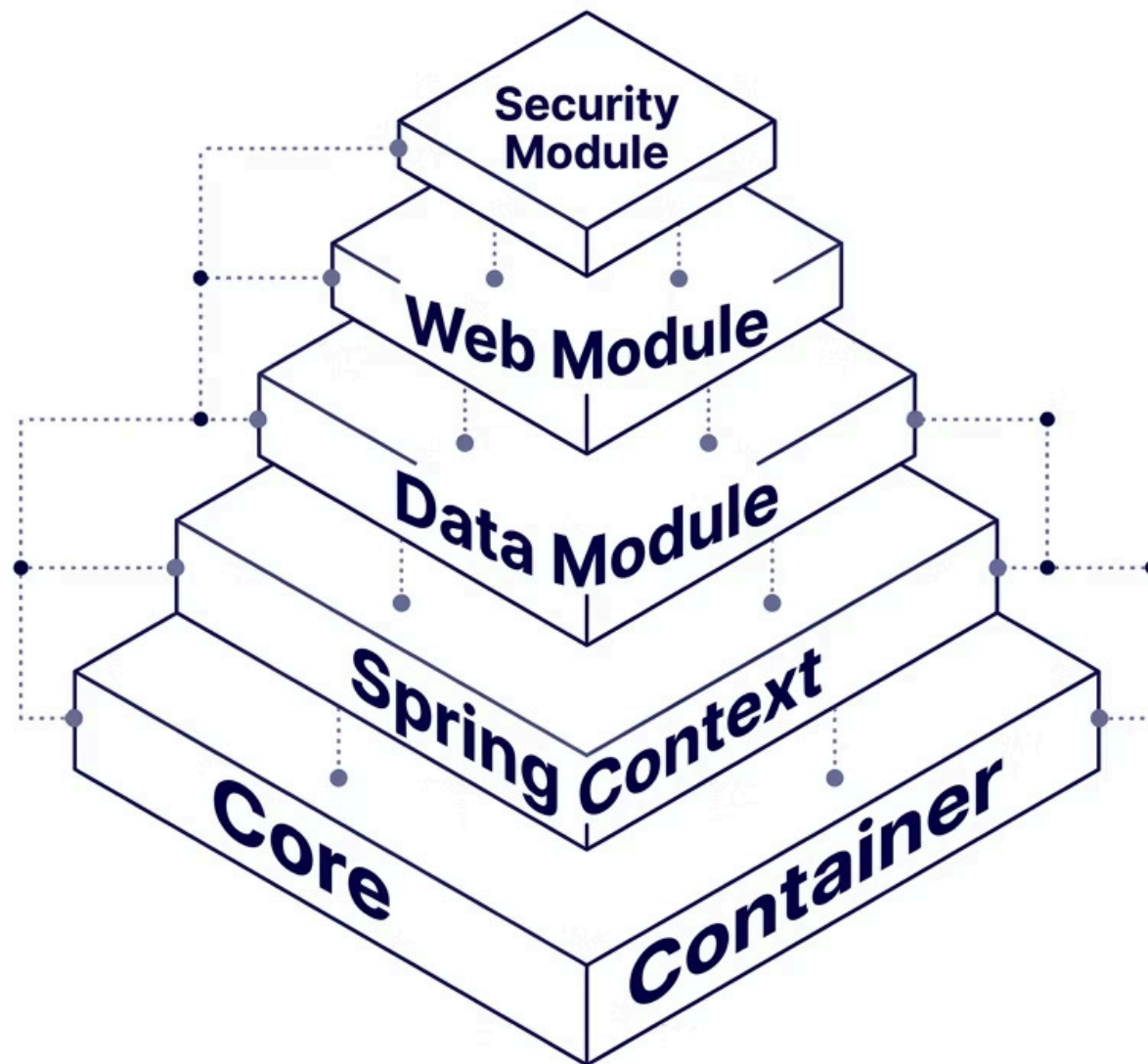


Faster Development

Less boilerplate, smart defaults, and powerful abstractions mean developers can deliver features more quickly and confidently.

Spring Architecture

Spring is composed of well-defined modules, each responsible for a specific concern. Together they form a complete enterprise application platform.



What is Spring Boot?

Spring Boot at a Glance

Spring Boot is an opinionated extension of the Spring Framework that eliminates heavy configuration and gets you up and running in minutes rather than hours.

Key Features



Auto Configuration

Automatically configures your application based on the dependencies present on the classpath — no manual XML required.



Embedded Server

Ships with an embedded Tomcat or Jetty server, so you can run your app as a standalone JAR without external deployment.



Faster Setup

Spring Initializr and sensible defaults mean a production-ready project skeleton is ready in seconds.

Where is Spring Used?

Spring powers some of the most demanding applications in the world. Its versatility makes it the framework of choice across a wide range of use cases.

REST APIs

Build robust, scalable HTTP APIs that power web and mobile frontends with Spring MVC and Spring Boot.

Enterprise Applications

Large-scale business systems rely on Spring's stability, security, and comprehensive ecosystem for mission-critical workloads.



Microservices

Spring Boot and Spring Cloud are the industry standard for building and orchestrating microservice architectures.

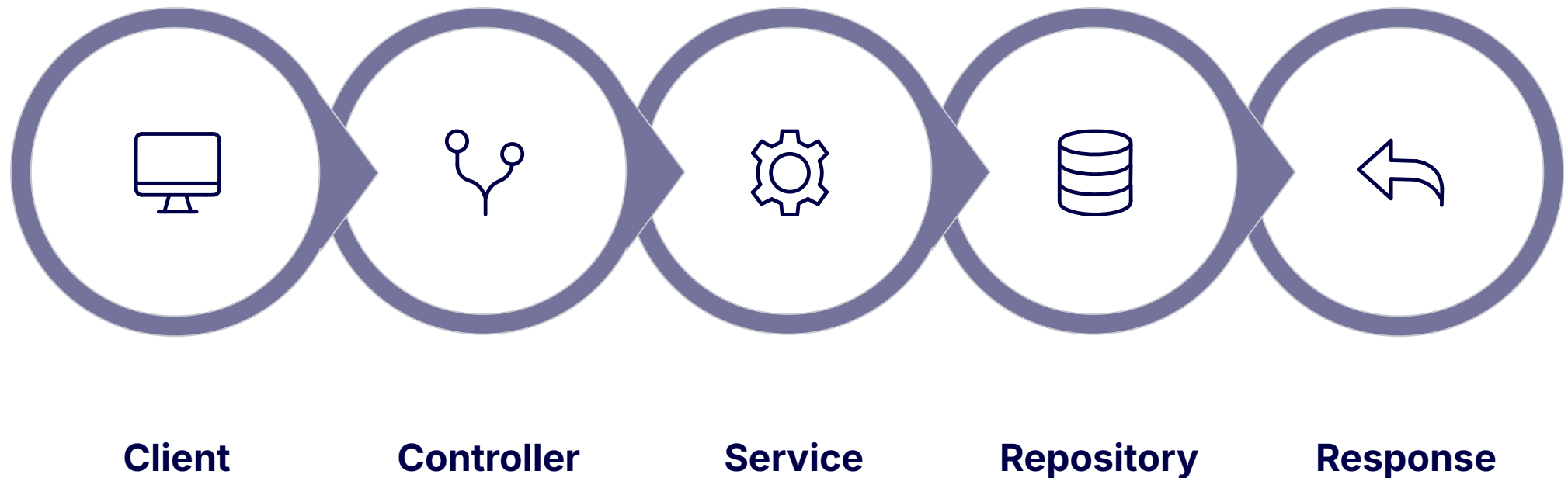


Cloud-Based Systems

Spring Cloud provides tools for service discovery, configuration management, and resilience in cloud-native deployments.

Example Flow — Simple View

Every request in a Spring application travels through a well-defined set of layers. Understanding this flow is fundamental to building and debugging Spring applications.



i This layered architecture enforces the **Separation of Concerns** principle — each layer has a single, well-defined responsibility, making the codebase easier to understand, test, and maintain.

In Your Training Journey

What You Will Learn

01

Spring Core

IoC container, Dependency Injection, and the foundational concepts that underpin the entire framework.

02

Spring Boot

Auto-configuration, embedded servers, and rapid application setup for modern backend development.

03

Spring Data JPA

Database access, ORM mapping, and repository patterns for clean data layer implementation.

04

REST APIs

Designing and building RESTful endpoints that power full stack applications end-to-end.

Where You Will Apply It

- **Backend Development Sprint** — Apply Spring Core and Spring Boot to build real backend services from scratch
- **Microservices Sprint** — Use Spring Boot and Spring Data JPA to design and deploy microservice-based architectures

📌 These skills form the backbone of your Full Stack Developer qualification and are directly applicable to real-world Cognizant projects.

Summary

Spring is not just a framework — it is the backbone of modern Java backend development. Here is what you need to remember.

Spring = Backbone of Java Backend

From small REST APIs to massive enterprise systems, Spring is the foundation that the Java ecosystem is built upon. Mastering it is non-negotiable for backend developers.

Promotes Clean Architecture

IoC, DI, and AOP enforce separation of concerns and loose coupling — resulting in codebases that are readable, testable, and maintainable over time.

Widely Used in Industry

Spring powers applications at companies of every size — from startups to Fortune 500 enterprises. It is one of the most in-demand skills in the Java ecosystem globally.

Essential for Full Stack Developers

As a Full Stack Developer, a solid understanding of Spring is what bridges your frontend skills with robust, production-ready backend systems.

Next Step — Hands-On

Theory only takes you so far. It is time to get your hands on the keyboard and build something real with Spring Boot.



Create Your First Spring Boot App

Use Spring Initializr to scaffold a new project, explore the auto-generated structure, and run your first Spring Boot application locally.



Build a Simple REST API

Create a `@RestController`, define endpoints with `@GetMapping` and `@PostMapping`, and test your API using a tool like Postman.



Connect to a Database

Integrate Spring Data JPA, configure a data source, define an `@Entity`, and use a `@Repository` to persist and retrieve data from a real database.



By the end of this hands-on exercise, you will have a fully functional Spring Boot application with a REST API backed by a database — the foundation of every modern Java backend.